

AWS 기본

inky4832@daum.net

6. EC2 기초

- EC2 기초 및 인스턴스, 유형
- 스토리지
- AMI
- 보안그룹
- SSH 이용한 EC2 접근
- Elastic IP
- EC2 사용자 데이터
- Auto Scaling
- ELB 및 모니터링(Cloudwatch)

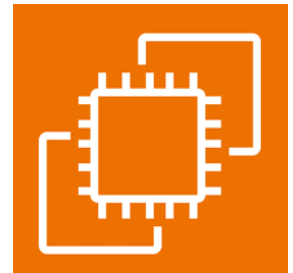
6.1 EC2 기초

개념

- Amazon Web Service 상에서 안정적이며 크기 조정 가능한 컴퓨팅 파워를 제공하는 웹 서비스
- 즉 사용자는 서버의 운영체제, CPU/메모리 사양, 디스크, 네트워크, 보안 설정 등을 직접 선택해서 원하는 형태의 서버를 빠르게 생성하고 필요 시 중지/재시작/삭제할 수 있음.

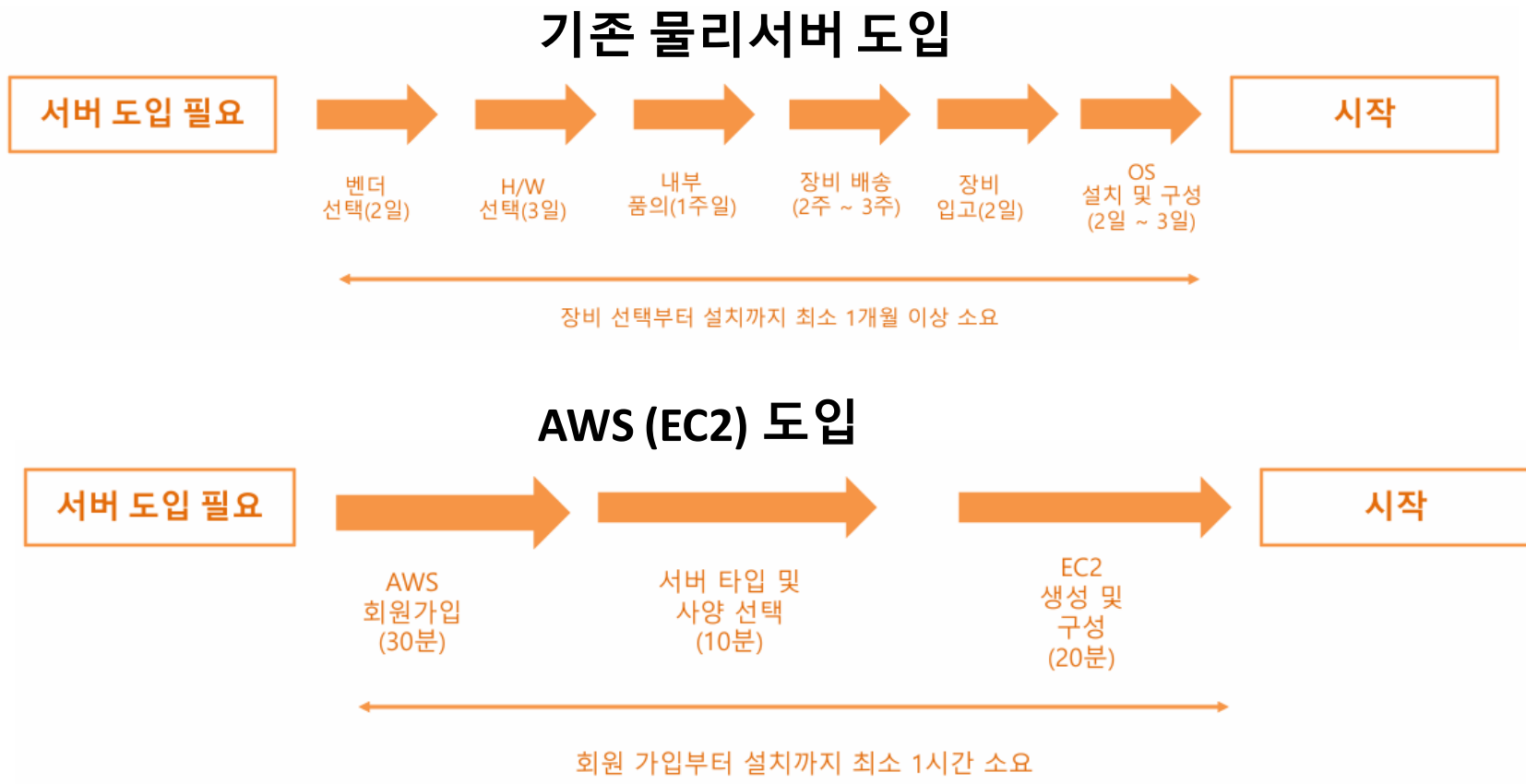
주요 특징

- 빠른 생성 및 유연한 확장
 - 필요한 순간에 서버를 바로 생성 가능
 - 현재 사용중인 인스턴스가 너무 작거나 크면 바로 타입을 변경해서 리사이징 가능
- 여러 AWS 서비스와 연동
- 다양한 운영체제 지원
- 표준화와 복제
 - 잘 구성된 인스턴스를 AMI로 저장하여 재사용
- 보안 제어
 - 보안 그룹을 통해 어떤 포트를 누구에게 허용할 것인지를 제어



6.1 EC2 기초

프로세스



6.1 EC2 기초

대표적인 사용 사례

- 웹 서버
- 데이터베이스 서버
- 머신 러닝
- 데이터 프로세싱
- 어플리케이션 백엔드 서버
- 기타 서버가 필요한 모든 작업

6.1 EC2 기초

핵심 개념

- 인스턴스 (Instance)
 - EC2에서 생성한 가상 서버 1대 의미
- AMI (Amazon Machine Image)
 - 인스턴스를 시작하기 위한 서버 이미지 템플릿
- 인스턴스 타입 (Instance Type)
 - 서버의 하드웨어 사양 클래스 의미
- 보안 그룹 (Security Group)
 - EC2 인스턴스에 연결되는 가상 방화벽
- 스토리지
 - EBS 볼륨

6.2 EC2 인스턴스

개념

- AWS 클라우드에서 실행되는 가상 서버(Virtual Server).
- 즉, 클라우드에서 사용하는 서버 한 대 의미.
- 하나의 가용영역(AZ)에 존재

활용

- 웹 서버
- 데이터베이스 서버
- 머신 러닝
- 데이터 프로세싱
- 어플리케이션 백엔드 서버
- 기타 서버가 필요한 모든 작업

6.2 EC2 인스턴스

구성요소 5가지

- AMI
 - 서버 운영체제 이미지
- Instance Type
 - CPU/메모리 성능
- Storage
 - 서버 디스크
- Security Group
 - 가상 방화벽
- Key Pair
 - 서버 접속 키

6.3 인스턴스 유형 (Instance Type)

개념

- EC2 서버의 성능을 결정하는 요소.
- 즉, 서버의 CPU/메모리/네트워크 성능을 결정.
- <https://aws.amazon.com/ko/ec2/instance-types/>

	범용				컴퓨팅 최적화				메모리 최적화					저장 최적화		
타입	t	m	A1	Mac	c	f	Inf	g	r	x	p	z	u-6tb1	h	i	d
설명	저렴한 범용	범용	ARM 기반	맥 기반	컴퓨팅 최적화	하드웨어 가속	머신러닝용	그래픽 최적화	메모리 최적화	메모리 최적화	그래픽 최적화	고주파수 컴퓨팅 워크로드	대용량 메모리 인스턴스	디스크쓰루풋 최적화	디스크 속도 최적화	디스크 최적화
예시	웹서버, DB	애플리케이션 서버	ARM 기반 에코 시스템 워크로드/웹서버 등		CPU 성능이 중요한 애플리케이션 /DB	유전 연구, 금융 분석, 빅 데이터 분석,	머신러닝	3D 모델링/인코딩	메모리 성능이 중요한 애플리케이션 /DB	Spark	머신러닝, 비트코인	EDA에 플레키이션	가상화 오버헤드를 줄여주는 베어메탈	하둡/맵리듀스	NoSql/데이터 웨어하우스	파일 서버/데이터 웨어하우스/하둡

6.3 인스턴스 유형 (Instance Type)

특징

- 인스턴스의 역할에 따라서 CPU, 메모리, 스토리지, 네트워크 등을 조합한 구성가능
- 각 인스턴스 유형별로 사용 목적에 따라 최적화
 - ex. 메모리 위주, CPU 위주, 그래픽카드 위주 등
- 유형 별로 이름 존재
 - ex. t유형, m유형 등
- 타입 별 세대별로 숫자 부여
 - ex. m5 는 m인스턴스의 5세대
- 아키텍처 및 프로세서/추가기술에 따라 접미사 지정
 - ex. c7gn 는 c 인스턴스 중 AWS Graviton프로세서를 사용(g) + Network Optimized(n) 적용

6.3 인스턴스 유형 (Instance Type)

인스턴스 크기

- 같은 인스턴스 패밀리에서 다양한 크기가 제공
- 인스턴스의 cpu 개수, 메모리 크기, 성능 등으로 크기가 결정
- 크기가 클수록
 - 더 많은 메모리
 - 더 많은 CPU
 - 더 많은 네트워크 대역폭
 - EBS와의 통신 가능한 대역폭

6.3 인스턴스 유형 (Instance Type)

제품 세부 정보 예

▼ 인스턴스 유형 [정보](#) | [조언 받기](#)

인스턴스 유형

t3.micro

프리 티어 사용 가능

패밀리: t3 2 vCPU 1 GiB 메모리 현재 세대: true

온디맨드 Windows 기본 요금: 0.0222 USD 시간당 온디맨드 Linux 기본 요금: 0.013 USD 시간당

온디맨드 RHEL 기본 요금: 0.0418 USD 시간당 온디맨드 SUSE 기본 요금: 0.013 USD 시간당

온디맨드 Ubuntu Pro 기본 요금: 0.0165 USD 시간당

☐ 모든 세대

인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.



인스턴스 유형 (1/735+) [조언 받기](#)

🔍 Find resources by attribute or tag

	인스턴스 유형 ▼	vCPU ▼	아키텍처 ▼	메모리(GiB) ▼	스토리지(GB) ▼	스토리지 유형 ▼	네트워크 성능
☐	t2.nano	1	i386, x86_64	0.5	-	-	Low to Moderate
☐	t2.micro	1	i386, x86_64	1	-	-	Low to Moderate
☐	t2.small	1	i386, x86_64	2	-	-	Low to Moderate
☐	t2.medium	2	i386, x86_64	4	-	-	Low to Moderate
☐	t2.large	2	x86_64	8	-	-	Low to Moderate
☐	t2.xlarge	4	x86_64	16	-	-	Moderate
☐	t2.2xlarge	8	x86_64	32	-	-	Moderate
☐	t3.nano	2	x86_64	0.5	-	-	Up to 5 Gigabit
☒	t3.micro	2	x86_64	1	-	-	Up to 5 Gigabit
☐	t3.small	2	x86_64	2	-	-	Up to 5 Gigabit
☐	t3.medium	2	x86_64	4	-	-	Up to 5 Gigabit
☐	t3.large	2	x86_64	8	-	-	Up to 5 Gigabit
☐	t3.xlarge	4	x86_64	16	-	-	Up to 5 Gigabit
☐	t3.2xlarge	8	x86_64	32	-	-	Up to 5 Gigabit

6.3 인스턴스 유형 (Instance Type)

인스턴스 유형 읽는 방법



6.3 인스턴스 유형 (Instance Type)

AWS Management Console 화면

▼ 인스턴스 유형 [정보](#) | [조언 받기](#)

인스턴스 유형

t3.micro

프리 티어 사용 가능

패밀리: t3 2 vCPU 1 GiB 메모리 현재 세대: true

온디맨드 Windows 기본 요금: 0.0222 USD 시간당 온디맨드 Linux 기본 요금: 0.013 USD 시간당 ▼

온디맨드 RHEL 기본 요금: 0.0418 USD 시간당 온디맨드 SUSE 기본 요금: 0.013 USD 시간당

온디맨드 Ubuntu Pro 기본 요금: 0.0165 USD 시간당

☒ 모든 세대

[인스턴스 유형 비교](#)

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

6.4 스토리지 (EBS: Elastic Block Store)

개념

- AWS에서 제공하는 EC2 인스턴스용 블록 스토리지 서비스임.
- 쉽게 말해 EC2 인스턴스의 하드 디스크임.

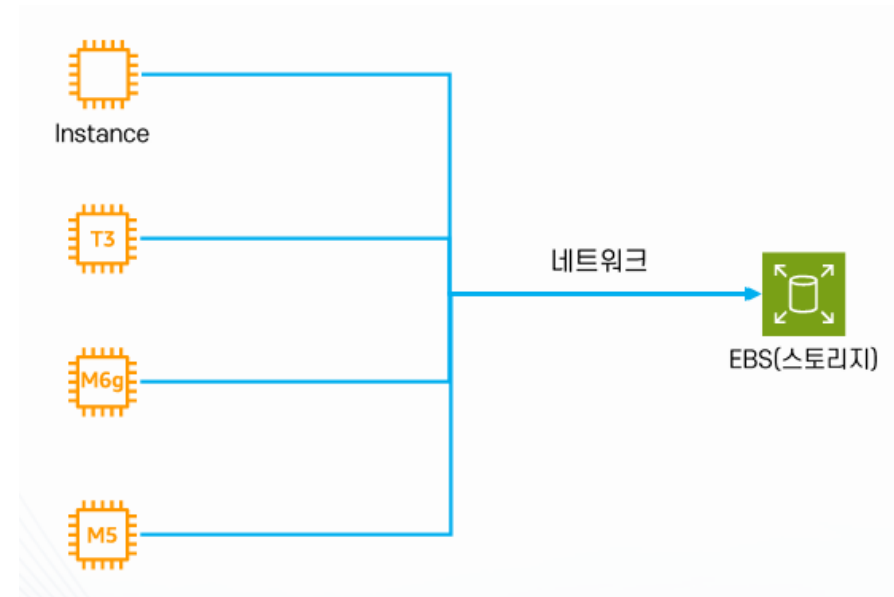
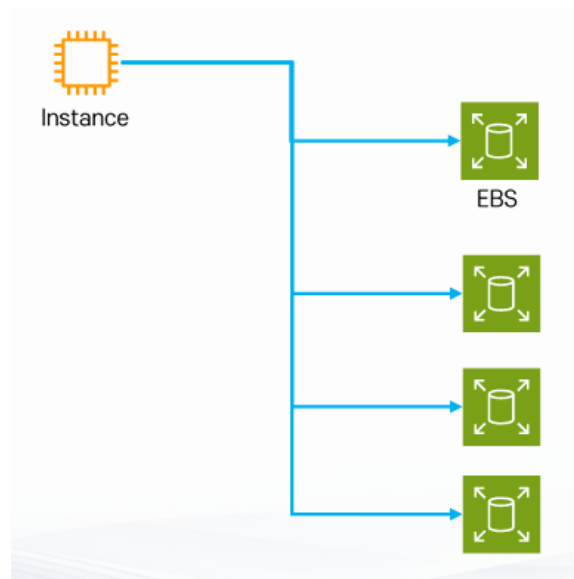
특징

- EBS는 네트워크 드라이브임 (EC2 인스턴스에 연결해서 사용)
- EC2 인스턴스가 종료되어도 계속 유지 가능
 - 단, root 볼륨으로 설정 시 EC2가 종료되면 같이 삭제됨
- 용량을 범위에 따라 자유롭게 설정 가능
- 하나의 EBS를 여러 EC2에 장착 가능
- EC2 인스턴스와 동일한 가용영역에 존재
 - 즉, 하나의 가용영역 안에서만 존재
 - us-east-1a에서 생성된 경우 us-east-1b에서 사용 불가
- 가용영역 안에 자동으로 분산 저장
- 스냅샷 백업 가능



6.4 스토리지 (EBS: Elastic Block Store)

연결(attach) 유형



6.4 스토리지 (EBS: Elastic Block Store)

볼륨 유형

타입	범용타입	프로비전된 IOPS	쓰루풋 최적화 HDD	Cold HDD	마그네틱
이름	GP	IO	ST	SC	Standard
용량	1GB~16TB	4GB~16TB	500GB~16TB	500GB~16TB	1GB~1TB
사용	일반 범용	IOPS가 중요한 어플리케이션/데이터베이스	쓰루풋이 중요한 어플리케이션/하둡/OLAP 데이터베이스 등	파일 저장소	백업/비 주기적인 데이터 액세스
MAX IOPS	16,000	64,000	500	250	40~200

6.4 스토리지 (EBS: Elastic Block Store)

스냅샷 (Snapshot)

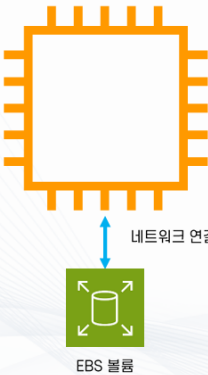
- EBS의 특정 시점을 저장한 이미지
 - EBS의 백업 용도로 활용
- 증분식
 - 바뀐 부분만 저장 (비용 최적화)
- S3에 저장
 - 99.999999999 % 내구성 (11 Nine)
- 암호화 가능
- 다른 계정 등과 공유 가능

6.4 스토리지 (EBS: Elastic Block Store)

스냅샷 생성 시 EC2 인스턴스 저장 유형

- 주요 옵션 2가지
 - Amazon EBS
 - Instance Store

EBS



네트워크 연결

EBS 볼륨

인스턴스 스토어



인스턴스 스토리지 볼륨

스냅샷 생성 정보

EBS 볼륨의 특정 시점 스냅샷을 생성하여 새 볼륨 또는 데이터 백업의 기준으로 사용합니다. 개별 볼륨에서 스냅샷을 생성하거나 인스턴스에 연결된 모든 볼륨에서 다중 볼륨 스냅샷을 생성할 수 있습니다.

소스

리소스 유형 | 정보

☒ 볼륨

특정 볼륨에서 스냅샷을 생성합니다.

☐ 인스턴스

인스턴스에서 다중 볼륨 스냅샷을 생성합니다.

볼륨 ID

스냅샷이 생성될 볼륨입니다.

볼륨 선택 🔄

스토리지 유형

볼륨에 사용되는 스토리지 유형입니다.

EBS 볼륨은 EC2 인스턴스의 수명과 독립적으로 유지되는 블록 수준 스토리지 볼륨이므로 데이터 손실 없이 나중에 인스턴스를 중지했다가 다시 시작할 수 있습니다. 또한 한 인스턴스에서 EBS 볼륨을 분리하고 다른 인스턴스에 연결할 수도 있습니다. EBS 볼륨은 인스턴스의 사용 비용과 별도로 청구됩니다.

인스턴스 스토어 볼륨은 호스트 컴퓨터에 물리적으로 연결됩니다. 이러한 볼륨은 인스턴스의 수명 동안에만 유지되는 임시 블록 스토리지를 제공합니다. 인스턴스를 중지하거나, 최대 절전 모드로 전환하거나, 종료하면 인스턴스 스토어 볼륨의 데이터가 손실됩니다. 인스턴스 유형에 따라, 사용 가능한 인스턴스 스토어 볼륨의 크기 및 수량과 인스턴스 스토어 볼륨에 사용되는 하드웨어 유형이 결정됩니다. 인스턴스 스토어 볼륨은 인스턴스 사용 비용의 일부로 포함됩니다.

6.4 스토리지 (EBS: Elastic Block Store)

AWS Management Console 화면

▼ 스토리지 구성 정보

1x 8 GiB gp3

루트 볼륨, 3000IOPS, 암호화되지 않음

새 볼륨 추가

⌚ 백업 정보를 보려면 새로 고침 클릭

할당된 태그에 따라 Data Lifecycle Manager 정책으로 인스턴스를 백업할지 여부가 결정됩니다.

0 x 파일 시스템

6.4 스토리지 (EBS: Elastic Block Store)

▼ 스토리지(볼륨) 정보

단순

EBS 볼륨

세부 정보 숨기기

▼ 볼륨 1 (AMI 루트)

스토리지 유형 | 정보

EBS

디바이스 이름 - 필수 | 정보

/dev/xvda

스냅샷 | 정보

snap-0cb9bf1e783385093

크기(GiB) | 정보

8

볼륨 유형 | 정보

gp3 ▼

IOPS | 정보

3000

종료 시 삭제 | 정보

예 ▼

암호화됨 | 정보

암호화되지 않음 ▼

KMS 키 | 정보

선택 ▼

이 볼륨에서 암호화가 설정된 경우에만 KMS 키를 적용할 수 있습니다.

처리량 | 정보

125

볼륨 초기화 속도 - 신규, 선택 사항 | 정보

값 입력

최소: 100MiB/s, 최대: 300MiB/s. 추가 요금 적용 ↗

새 볼륨 추가

6.5 AMI (Amazon Machine Image)

개념

- EC2 인스턴스를 생성할 때 사용하는 서버 원본 이미지.
- EC2 인스턴스가 실제로 실행되는 가상서버라면, AMI는 그 서버를 만들기 위한 설계도 + 운영체제 이미지 + 기본 설정 묶음 이라고 보면 됨.
- 필요시 커스텀 AMI 생성 가능

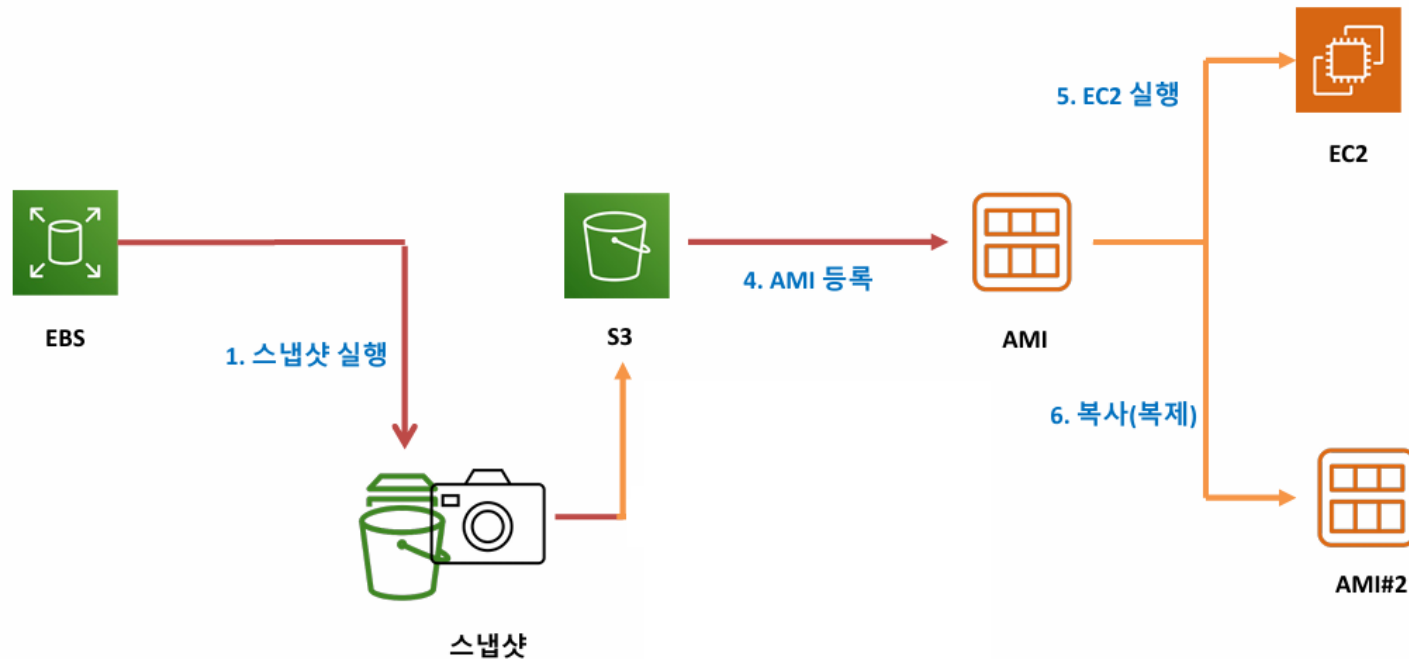
구성요소

- 운영체제
 - ex. Amazon Linux, **Ubuntu**, Windows 등
- 부팅에 필요한 정보
 - 인스턴스 시작용 설정
- 기본 소프트웨어
 - 필요 시 웹서버, 에이전트, 런타임 등
- 블록 디바이스 매핑
 - 어떤 디바이스를 연결할지에 대한 정보

6.5 AMI (Amazon Machine Image)

인스턴스 저장유형에 따른 AMI 생성

- EBS Snapshot 형태로 만들어지고 내부적으로 Amazon S3에 보관됨



6.5 AMI (Amazon Machine Image)

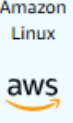
AWS Management Console 화면

▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI에는 인스턴스의 운영 체제, 애플리케이션 서버, 애플리케이션이 포함됩니다. 아래에 적합한 AMI가 보이지 않는 경우 검색 필드를 사용하거나 더 많은 AMI 찾아보기를 선택하세요.

🔍 수천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색

Quick Start



Amazon Linux



macOS



Ubuntu



ubuntu®



Windows



Red Hat



SUSE Linux



Debian



더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

Amazon Machine Image(AMI)

Amazon Linux 2023 kernel-6.1 AMI
 ami-0ecfd1c8ae01aec (64비트(x86), uefi-preferred) / ami-055751883cc1be227 (64비트(Arm), uefi)
 가상화: hvm ENA 활성화됨: true 루트 디바이스 유형: ebs

설명

Amazon Linux 2023(kernel-6.1)은 5년의 장기 지원을 제공하는 최신 범용 Linux 기반 OS입니다. AWS에 최적화되어 있으며 클라우드 애플리케이션을 개발 및 실행할 수 있는 안전하고 안정적인 고성능 실행 환경을 제공하도록 설계되었습니다.

Amazon Linux 2023 AMI 2023.10.20260302.1 x86_64 HVM kernel-6.1

아키텍처

64비트(x86) ▼

부트 모드

uefi-preferred

AMI ID

ami-0ecfd1c8ae01aec

게시 날짜

2026-03-03

사용자 이름 ⓘ

ec2-user

확인된 공급 업체

6.6 보안그룹 (Security Group)

개념

- EC2 인스턴스의 가상 방화벽 역할을 담당하는 서비스
- 즉, EC2 서버에 어떤 네트워크 트래픽을 허용할지 결정
- 자동으로 default 보안그룹이 제공

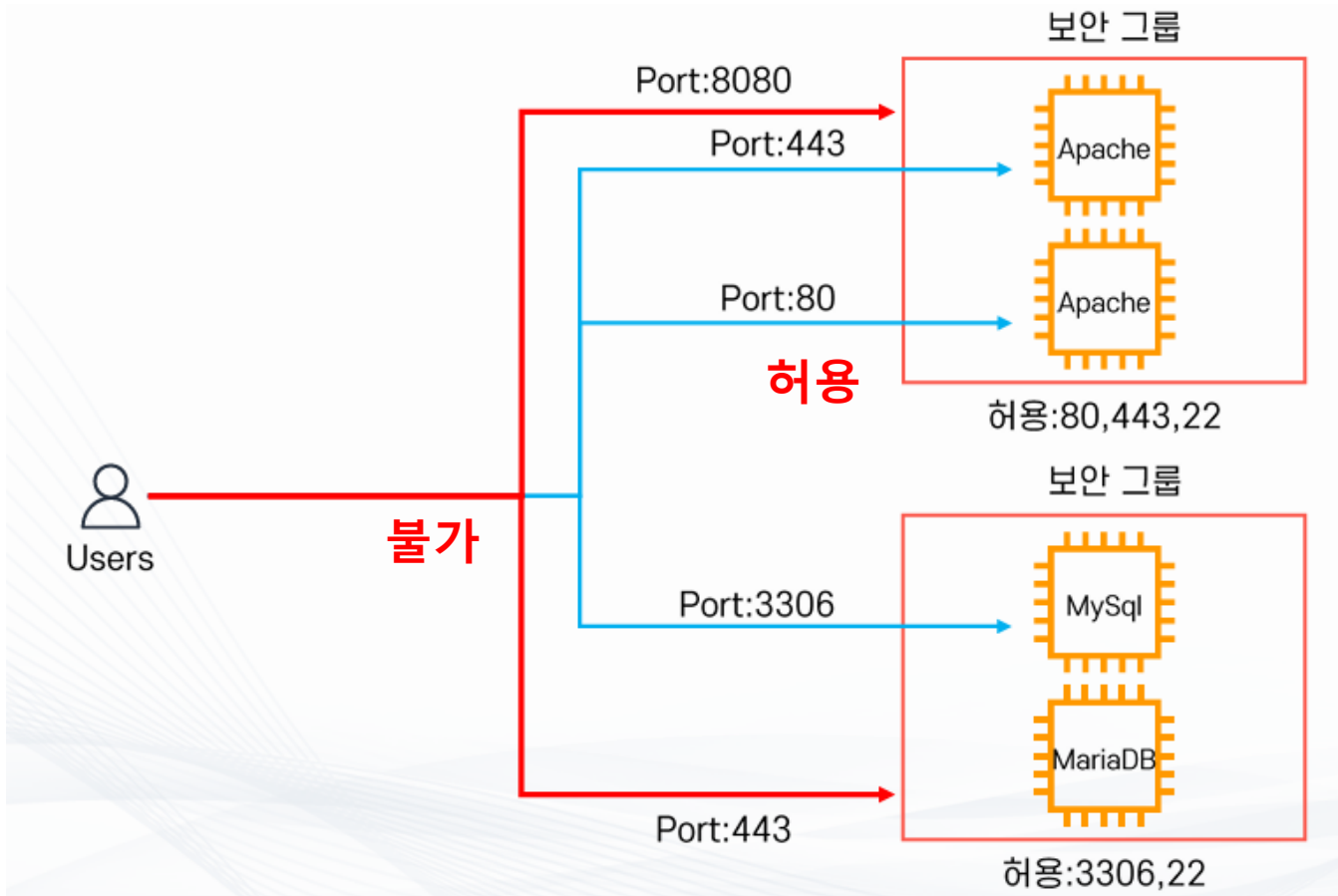
특징

- IP 및 port 허용
 - 기본적으로 모든 포트는 비활성화
 - 선택적으로 트래픽이 지나갈 수 있는 Port와 Source를 설정
- 인스턴스 단위
 - 하나의 인스턴스에 하나 이상의 보안그룹 설정 가능

두 가지 규칙

규칙	의미
Inbound Rule	들어오는 트래픽 허용
Outbound Rule	나가는 트래픽 허용, 기본은 모든 트래픽 허용

6.6 보안그룹 (Security Group)



6.7 실습 1

EC2에 웹서버 설치

- Amazon EC2 프로비전
 - t3.micro 타입 (프리티어)
- AMI 선택
 - **ubuntu (Ubuntu Server 24.04 LTS)**
- 보안그룹 설정
 - 80, 22 포트 허용
- 아파치 설치 및 설정
- 웹 서버 확인
- AMI 생성
- 기존 인스턴스 종료
- AMI로부터 새로운 EC2 인스턴스 생성
- 신규 인스턴스 확인 및 종료

6.7 실습 2

EC2에 EBS 추가

- Amazon EC2 프로비전
 - t3.micro 타입 (프리티어)
- AMI 선택
 - ubuntu (Ubuntu Server 24.04 LTS)
- 보안그룹 설정
 - 80, 22 포트 허용
- EBS 볼륨 추가
 - 인스턴스 생성 후 나중에 추가
 - 크기는 2Gib
- EC2 인스턴스에 EBS 볼륨 연결
 - 생성된 볼륨선택 > 작업 > 볼륨 연결 > 인스턴스 선택
- 해당 인스턴스 삭제 후 EBS 볼륨 존재 확인
- 추가된 EBS 볼륨은 스냅샷 실습 때 사용

6.7 실습 3

EBS Snapshot 생성 후 다른 Region에 복사 및 볼륨생성

- Snapshot 생성
 - EBS 선택 > 작업 > 스냅샷 생성 > 설명: DemoSnapshot 입력 > 스냅샷 생성 클릭
 - 리소스 유형은 볼륨을 선택
- 다른 Region 에 복사
 - 스냅샷 선택 > 작업 > 스냅샷 복사 클릭 > 대상 리전 선택 > 스냅샷 복사 클릭
 - 재해 발생에 대비한 백업용
- 대상 리전 스냅샷에서 볼륨 생성
 - 대상 리전 선택 > 스냅샷 선택 > 스냅샷에서 볼륨 생성 > 가용영역 선택
 - 가용영역은 반드시 EC2 인스턴스와 동일한 필요 없음
- 볼륨 및 스냅샷 삭제
 - 대상 리전 및 현재 리전에서 모두 볼륨 및 스냅샷 삭제

6.8 EC2 인스턴스 접근

개념

- 생성된 EC2 인스턴스에 접근하는 다양한 방법이 지원

종류

- SSH 연결 (*)
- EC2 인스턴스 연결 (*)
- Session Manager
- EC2 직렬 콘솔

연결 정보

브라우저 기반 클라이언트를 사용하여 인스턴스에 연결합니다.

인스턴스 ID i-05a9e489c10661c41 (demo-my-ec2)	VPC ID vpc-03bed3152859a7cdc
---	--

EC2 인스턴스 연결 | SSM Session Manager | SSH 클라이언트 | EC2 직렬 콘솔

6.9 SSH (Secure Shell) 연결

개념

- SSH는 원격 서버에 안전하게 접속하기 위한 네트워크 프로토콜임 (기본 port: 22)
- SSH Client 이용하여 연결 ex. MobaXterm, putty

지원

- 접속 구간 암호화
- 사용자 인증
- 명령 실행
- 파일 전송 지원

필요 항목

- EC2 Public IP 또는 DNS (접속 대상 주소)
- SSH Key Pair (인증용 키)
- 보안그룹 (22번 포트 허용)
- 사용자 계정명 (Ubuntu는 **ubuntu**로 고정, Amazon Linux는 ec2-user 로 고정)

6.10 실습

EC2에 SSH로 연결

- MobaXterm 설치
- EC2 인스턴스 프로비전
 - 키페어 생성 (SSH key)
- MobaXterm 실행
 - pem 설정
- EC2 인스턴스 로그인
 - **ubuntu** 계정
- EC2 인스턴스 삭제

6.11 EC2 생명 주기

개념

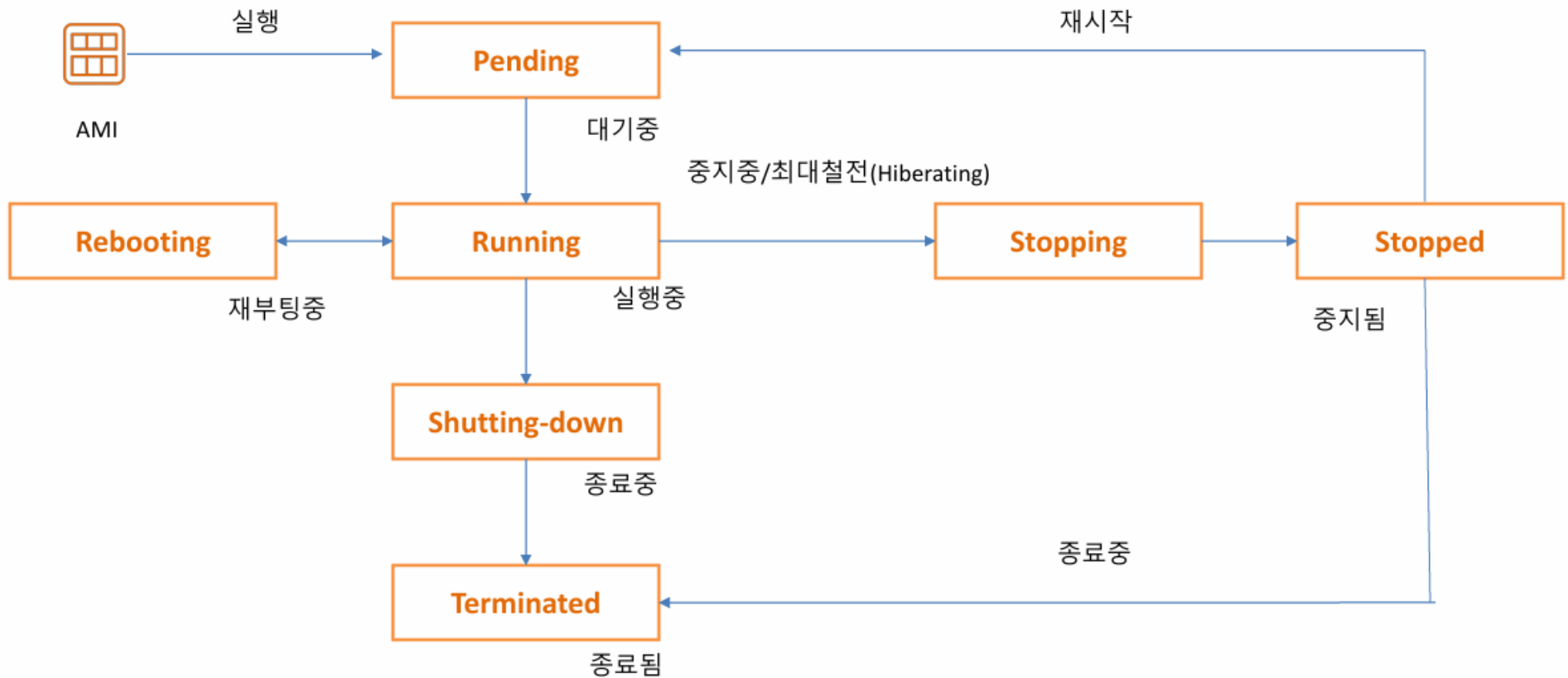
- EC2 인스턴스가 생성되어 실행되고 중지되거나 종료될 때까지 거치는 상태변화 과정을 의미함

상태

- pending : 생성 준비 중
- running : 실행 중
- stopping : 중지 처리 중
- stopped : 중지됨
- shutting-down : 종료 처리 중
- terminated : 완전 삭제

6.11 EC2 생명 주기

프로세스



6.11 EC2 생명 주기

Pending

- 인스턴스가 실행 준비 중인 상태로서 아직 서버 사용은 불가
 - 선택한 AMI로 부팅 준비
 - 선택한 인스턴스 타입에 맞는 하드웨어 할당
 - root 볼륨 연결
 - 네트워크 설정

Running

- 인스턴스가 정상적으로 실행 중인 상태로서 요금도 이때부터 발생 (초단위)
 - SSH 접속 가능
 - 웹 서버 실행 가능
 - 어플리케이션 배포 가능
 - 사용자가 실제 서버처럼 사용 가능

6.11 EC2 생명 주기

Stopping

- 인스턴스를 중지하는 중간 상태
 - 중지 중에는 EC2 인스턴스 요금만 미청구, 다른 구성요소(Elastic IP, EBS 등)은 청구
 - 중지 후 재시작시 퍼블릭 IP 변경됨

Stopped

- 인스턴스가 꺼져 있는 상태
 - SSH 접속 불가
 - 웹 서버 실행 불가
 - 어플리케이션 실행 안됨
 - 다시 start 가능

6.11 EC2 생명 주기

Shutting-down

- 인스턴스를 종료(terminate) 처리 중인 상태
 - 인스턴스를 삭제하기 위한 정리 작업을 진행
 - 요금 미청구

Terminated

- 인스턴스가 완전히 삭제된 상태
 - 다시 시작 불가
 - 복구 불가
 - 접속 불가

6.11 EC2 생명 주기

Stop vs Terminate

구분	Stop	Terminate
의미	서버 끄기	서버 삭제
다시 시작	가능	불가능
인스턴스 ID	유지	종료됨
EBS 루트 볼륨	유지	기본적으로 삭제
사용 요금	인스턴스 요금 중단	인스턴스 요금 중단

6.11 EC2 생명 주기

Reboot vs Stop/Start

구분	Reboot	Stop / Start
의미	운영체제 재부팅	인스턴스를 완전히 종료 후 다시 시작
서버 상태 변화	Running > Running	Running > Stopped > Pending > Running
물리 서버 변경 (인프라 변경)	변경되지 않음 (같은 Host 유지)	다른 호스트로 이동
public IP	유지	변경
Private IP	유지	유지
Elastic IP	유지	유지
Instance Store 데이터	유지	삭제
EBS 데이터	유지	유지
인스턴스 ID	유지	유지
사용 목적	OS 문제 해결, 서비스 재시작	인스턴스 상태 초기화, 인프라 문제 해결
요금	계속 과금	Stopped 상태에서는 미과금

6.11 EC2 생명 주기

고급 옵션

- 인스턴스 생성 시 또는 이후 설정 가능

인스턴스 자동 복구 | 정보

선택

종료 동작 | 정보

중지

최대 절전 중지 방식 | 정보

선택

종료 방지 | 정보

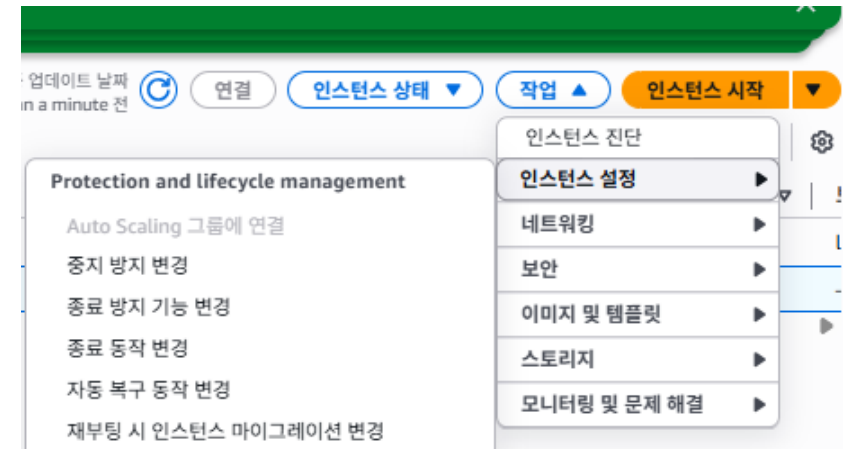
선택

중지 방지 | 정보

선택

세부 CloudWatch 모니터링 | 정보

선택



6.12 실습

EC2의 중지 및 재부팅

- 중지 후 재실행시 IP 주소가 변경되는 것을 확인
- 재부팅 시 IP 주소가 변경되지 않는 것을 확인
- EC2 삭제

6.13 ENI (Elastic Network Interface) 와 Elastic IP

개념

- ENI는 EC2의 가상의 네트워크 카드 (Virtual Network Card)
- 리눅스 기준으로 eth0, eth1, eth2 같은 네트워크 인터페이스

특징

- EC2와 분리 및 재연결 가능
- 하나의 EC2 인스턴스에 여러 개의 ENI를 연동 가능
 - 하나의 인스턴스가 한 개 이상의 IP를 보유 가능
- 인스턴스 유형 및 사이즈에 따라 최대 보유 가능한 IP주소가 결정
- 내부적으로 보안그룹은 ENI에 부착

6.13 ENI (Elastic Network Interface) 와 Elastic IP

ENI 구성요소

- 단순한 네트워크 카드가 아닌 다양한 정보를 포함

구성 요소	설명
Private IP	기본 Private IP
Secondary Private IP	추가 Private IP
Elastic IP	Public IP에 연결 가능
Mac Address	네트워크 고유 주소
Security Group	보안정책
Subnet	연결된 서브넷
Network Interface ID	eni-xxxx

6.13 ENI (Elastic Network Interface) 와 Elastic IP

Elastic IP (탄력적 IP)

- AWS에서 제공하는 고정 Public IP 주소
- EC2 중지/시작과 무관하게 항상 고정

특징

- ENI에 연결
 - 다른 EC2로 Elastic IP 이동 가능
- EC2 이외에 다른 서비스(ex. ALB)에도 사용
- 리전 단위
- 연결하지 않고 보유하기만 해도 비용 발생 (IPv4 비용)

6.14 실습

Elastic IP 사용

- EC2 생성
- Elastic IP 설정 및 EC2 연결
 - 퍼블릭 IPv4 주소 확인 ex. 13.124.219.47
 - 탄력적 IPv4 주소 확인 ex. [3.39.94.137](#)
 - Elastic IP 선택 > 탄력적 IP 주소 연결 > 인스턴스 선택 > 연결 클릭
 - EC2 인스턴스 화면에서 새로고침 후 퍼블릭 IPv4 확인 > 탄력적 IP로 변경
- EC2 중지 > 재시작
 - 중지된 상태에도 퍼블릭 IPv4 값이 유지됨
 - Elastic IP 변동 없음 확인
- Elastic IP 정리
 - 연결해제 > 릴리즈
- EC2 종료

6.15 EC2 User Data

개념

- EC2 인스턴스의 최초 실행 시 자동으로 실행되는 스크립트 또는 설정 데이터
- EC2 초기 설정을 자동화하기 위한 기능
- 별도 설정을 통해서 재부팅마다 실행하도록 설정도 가능

대표작업

- 웹 서버 설치
- 패키지 업데이트
- 어플리케이션 설치
- 환경 설정
- 스크립트 실행

6.15 EC2 User Data

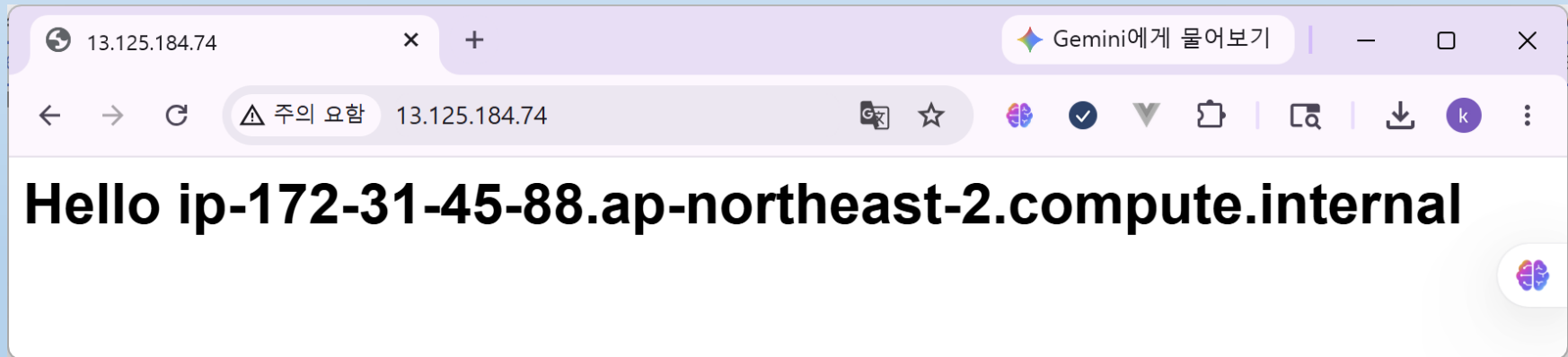
대표적인 스크립트 예

```
#!/bin/bash
apt update -y
apt install apache2 -y
systemctl start apache2
systemctl status apache2
echo "<h1>Hello $(hostname -f)</h1>" > /var/www/html/index.html
```

6.16 실습

EC2 User Data

- EC2 생성 + 스크립트 설정
 - 고급 세부 정보 > 사용자 데이터 항목
- 웹 브라우저에서 요청
 - 퍼블릭 IPv 4 DNS 또는 퍼블릭 IPv 4 주소 이용
- EC2 종료



6.17 EC2 권한 부여

개념

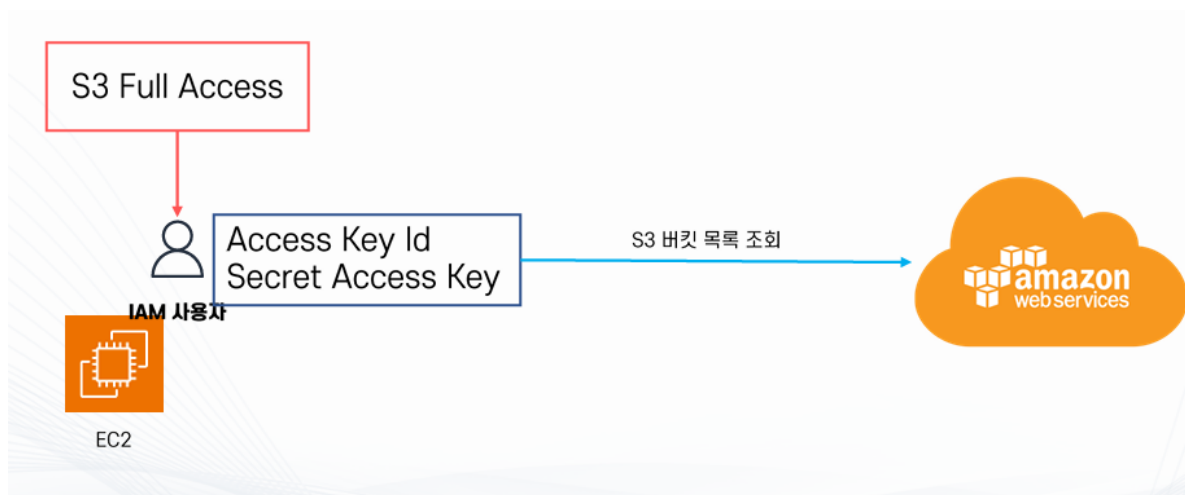
- EC2 인스턴스 자체가 S3, DynamoDB 같은 AWS 서비스에 접근할 권한 설정 방법.

방법 2가지

- IAM 자격증명(Credential) 이용
 - IAM 사용자를 생성하고 IAM 자격증명인 Access Key 발급받아 EC2에 등록해서 접근
 - aws configure 명령어를 통해서 자격증명을 ~/.aws/credentials 폴더에 저장
 - 관리가 어려움
 - 비추천 방식
- IAM 역할(Role) 이용
 - 권한이 부여된 IAM 역할(Role)을 만들고 EC2에 부여
 - 관리가 쉽고 교체가 쉬움
 - 추천 방식

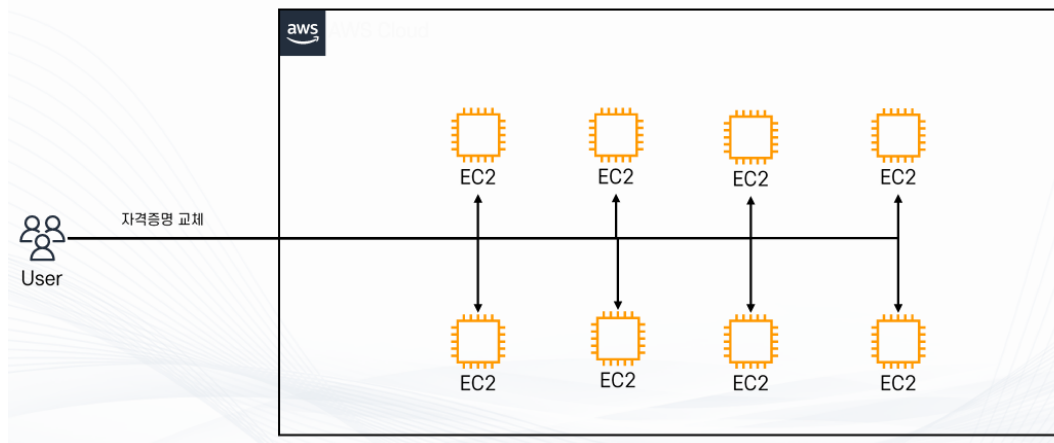
6.17 EC2 권한 부여

IAM 자격증명 (Credentials)



작업순서

1. IAM 사용자 생성
2. Access Key 생성
3. aws configure 로 설정
4. S3 접근



자격 증명 교체 시 개별적으로 각각 수정

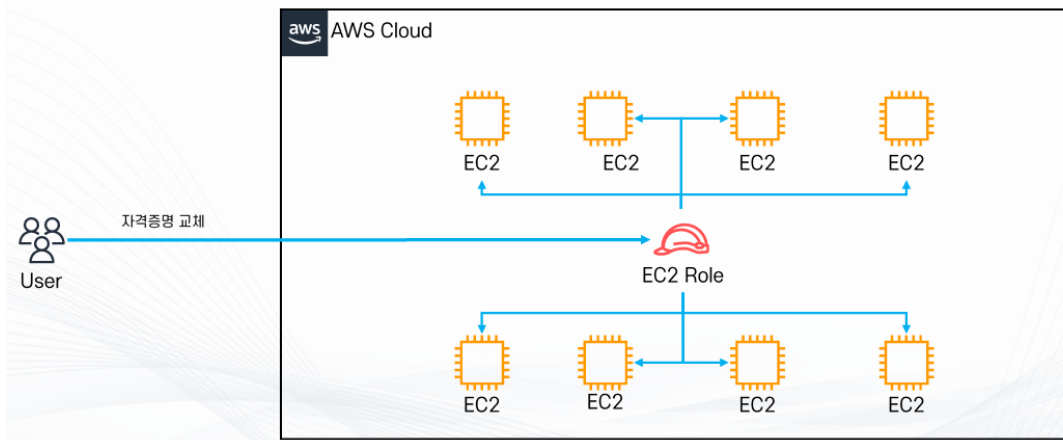
6.17 EC2 권한 부여

IAM 역할 (Role) - 추천 방식



작업순서

1. Role 생성
2. EC2에 Role 부여
3. S3 접근



자격 증명 교체 시 Role 만 수정

6.18 실습

IAM user 목록

- EC2 용 IAM Role 생성
 - 역할 > 역할생성 > AWS 서비스 + EC2 선택 > IAMReadOnlyAccess 권한정책 > 역할이름(demo-iam-role-for-ec2) > 역할생성 클릭
- EC2 생성
 - 고급 세부 정보 > IAM 인스턴스 프로파일 > 역할 선택

▼ 고급 세부 정보 [정보](#)

도메인 조인 디렉터리 [정보](#)

선택 [새 디렉터리 생성](#)

IAM 인스턴스 프로파일 [정보](#)

demo-iam-role-for-ec2
arn:aws:iam::585097637103:instance-profile/demo-iam-role-for-ec2 [새 IAM 프로파일 생성](#)

호스트 이름 유형 [정보](#)

IP 이름

DNS 호스트 이름 [정보](#)

6.18 실습

IAM user 목록

- 연결 > 'EC2 인스턴스 연결' 클릭 > ubuntu에 awscli 설치
 - `sudo -s`
 - `root # apt update -y`
 - `root # apt install unzip curl -y`
 - `root # curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"`
 - `root # unzip awscliv2.zip`
 - `root # ./aws/install`
 - `root # aws --version`
- EC2 인스턴스 연결
 - `root>aws sts get-caller-identity` # 현재 AWS CLI가 어떤 AWS 계정/권한으로 인증되어 있는지 확인
 - `root> aws iam list-users`

6.18 실습

```
root@ip-172-31-37-153:/home/ubuntu# aws sts get-caller-identity
{
  "UserId": "AROAYQOUJNTXXXM6VVZ6B:i-0c123ac06f61b6ca0",
  "Account": "585097637103",
  "Arn": "arn:aws:sts::585097637103:assumed-role/demo-iam-role-fo
}
root@ip-172-31-37-153:/home/ubuntu# aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "admin",
      "UserId": "AIDAYQOUJNTXW4YHYCH7R",
      "Arn": "arn:aws:iam::585097637103:user/admin",
      "CreateDate": "2026-05-09T01:21:58+00:00",
      "PasswordLastUsed": "2026-05-09T03:59:40+00:00"
    },
    {
      "Path": "/",
      "UserName": "demo-iam-user",
      "UserId": "AIDAYQOUJNTXRSFOD3S2C",
      "Arn": "arn:aws:iam::585097637103:user/demo-iam-user",
      "CreateDate": "2026-05-09T05:16:03+00:00"
    }
  ]
}
```

6.18 실습

IAM user 목록

- EC2 인스턴스에서 역할 분리
 - 작업 > 보안 > IAM 역할 수정 > IAM 역할 > IAM 역할 없음 선택
 - root> aws iam list-users # 재요청

```
root@ip-172-31-37-153:/home/ubuntu# aws iam list-users
aws: [ERROR]: An error occurred (NoCredentials): Unable to locate credentials.
```

- EC2 삭제

6.19 Auto Scaling

개념

- 시스템의 트래픽에 따라 컴퓨팅 자원을 자동으로 늘리거나 줄이는 기술
 - ex. 트래픽이 증가하면 서버 자동 증가
트래픽이 감소하면 서버 자동 감소

목적

- 정확한 수의 EC2 인스턴스 보유 보장
 - 그룹의 최소 인스턴스 숫자 및 최대 인스턴스 숫자 이용
- 고가용성 (High Availability)
- 성능 유지
- 비용 최적화

특징

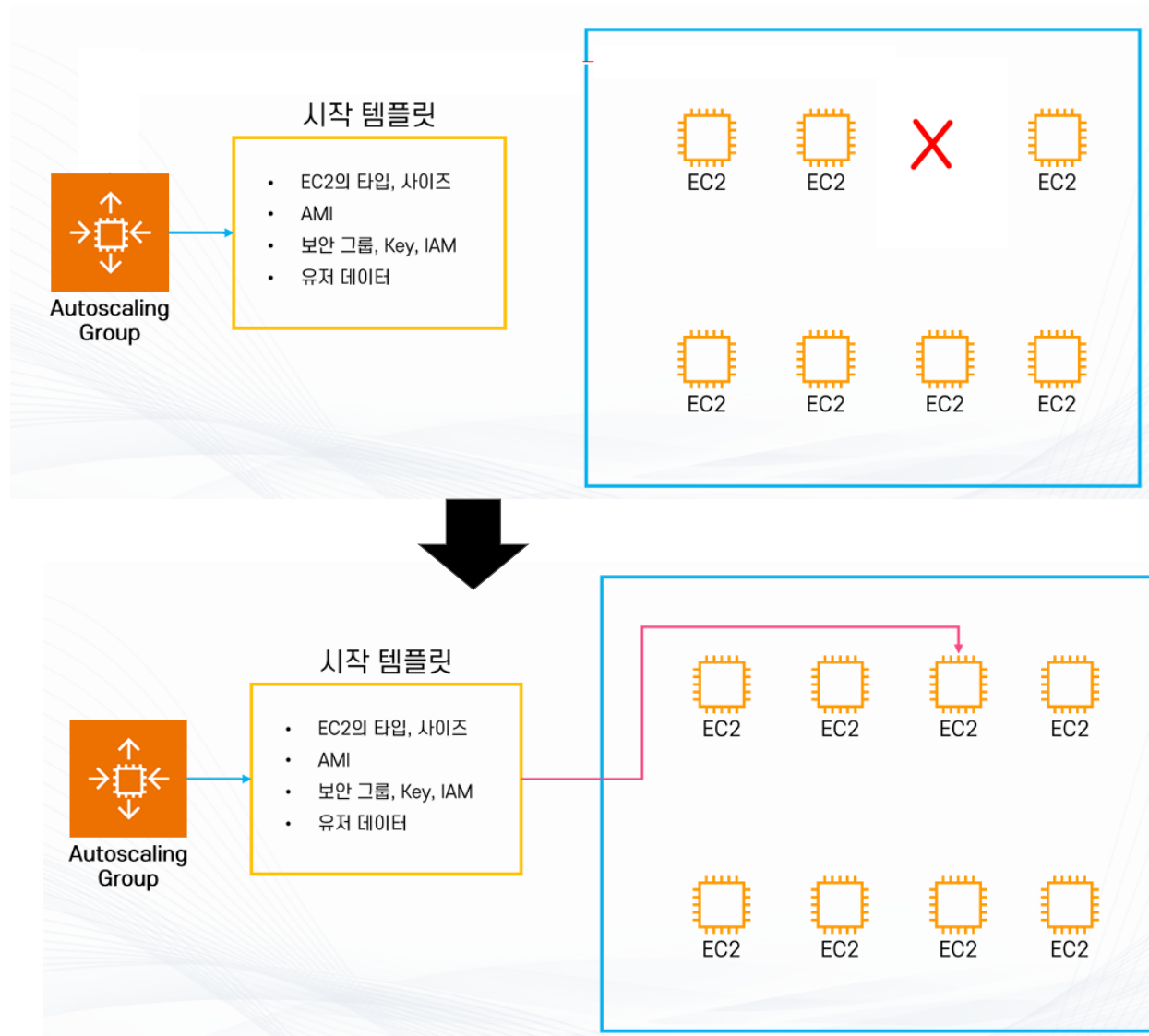
- 다양한 스케일링 정책
 - ex. CPU 부하에 따른 스케일링, 특정 시간에 맞춘 스케일링
- 가용 영역에 인스턴스가 골고루 분산될 수 있도록 인스턴스 분배

6.19 Auto Scaling

주요 구성요소

- Launch Template
 - 새로운 EC2 인스턴스를 만들 때 사용하는 템플릿(설계도)
 - AMI, User Data, IAM Role 포함 가능
- Auto Scaling Group (ASG)
 - 여러 EC2 인스턴스를 하나의 그룹으로 묶어 개수유지, 자동 확장/축소, 장애 대체를 담당
 - 최소 개수, 목표 개수, 최대 개수 설정
- CloudWatch
 - AWS의 모니터링 서비스
 - Auto Scaling의 확장/축소 판단의 근거가 되는 metric 제공
- Load Balancer
 - 여러 EC2 인스턴스로 들어오는 요청을 분산하는 역할

6.19 Auto Scaling



6.20 실습

Auto Scaling

- EC2 시작 템플릿 생성
 - 기존 모든 EC2 인스턴스 종료
 - EC2 > 인스턴스 > 시작 템플릿 > 시작 템플릿 생성 클릭
 - 이름: demo-my-ec2-template
 - AMI: **ubuntu (Ubuntu Server 24.04 LTS)**
 - 인스턴스 유형: t3-micro
 - 키 페어: demo-my-ec2-keypair
 - 네트워크 설정: 서브넷과 가용영역 모두 포함하지 않음 선택.
 - 보안그룹: 새로 생성 (demo-my-security-group),
 - 인바운드 규칙: 모든 트래픽/위치무관 (0.0.0.0/0) (실습용)
 - 사용자 데이터 : 아파치 웹서버 실습스크립트

6.20 실습

■ Auto Scaling Group 생성

- EC2 > Auto Scaling > Auto Scaling 그룹
- 이름: demo-my-auto-scaling-group
- 시작 템플릿: demo-my-ec2-template , 버전관리 가능
- VPC: default
- 가용영역 및 서브넷: 4 개 모두 선택 (2개만 사용가능, 경고는 무시)
- 로드밸런서: '로드밸런서 없음' 선택
- 원하는 용량(초기용량),최소용량, 최대용량: 3, 1, 3
- 추가설정 > 모니터링 > CloudWatch내에서 그룹 지표 수집 활성화 체크
- 태그: Name, demo-my-asg

Auto Scaling 그룹 (1) [Info](#) 최근 업데이트 1분 전 시작 구성 시작 템플릿 ↗ 작업 ▼ Auto

🔍 Auto Scaling 그룹 검색

☐	이름	상태 - 새로 만들기 ▼	시작 템플릿/구성 ↗ ▼	인스턴스 ▼	Instance health - 새...
☐	demo-my-auto-scaling-group	✔ At desired capacity	demo-my-ec2-template 버전 기본값	3	✔ 3/3 Healthy

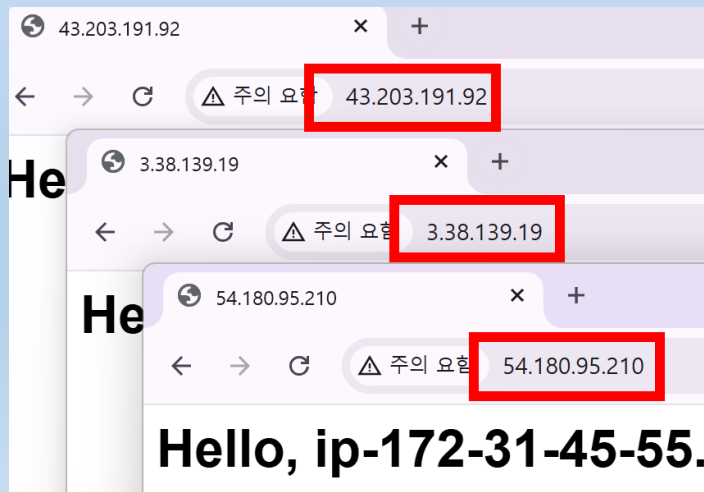
Auto Scaling 그룹: demo-my-auto-scaling-group

세부 정보 | 통합 | Automatic scaling | **인스턴스 관리** | 인스턴스 새로 고침 | 활동 | 모니터링 | 태그

인스턴스 (3)

Q 인스턴스 필터링

☐	인스턴스 ID	수명 주기	인스턴스 유형	가중치 기...	시작 템플...	가용 영역	상태
☐	i-055c72cb80cd94363	InService	t3.micro	-	demo-my-ec2-ter	apne2-az3 (a...	✓ Healthy
☐	i-080e77112f51c1177	InService	t3.micro	-	demo-my-ec2-ter	apne2-az2 (a...	✓ Healthy
☐	i-0bfac8d1e8cc1a044	InService	t3.micro	-	demo-my-ec2-ter	apne2-az4 (a...	✓ Healthy



6.21 ELB (Elastic Load Balancer)

개념

- 여러 서버로 들어오는 트래픽을 자동으로 분산하는 AWS 서비스
- 즉, 사용자 요청을 한 서버에 집중시키지 않고 여러 서버로 나누어 전달함

목적

- 서버 부하 분산
- 고가용성(High Availability)
- 장애 서버 자동 제외
- Auto Scaling 연동

6.21 ELB (Elastic Load Balancer)

종류

- **ALB (Application Load Balancer)**
 - HTTP/HTTPS 기반 로드밸런서 (Layer 7)
 - 웹 서비스에 가장 많이 사용
- **NLB (Network Load Balancer)**
 - TCP/UDP 기반 로드밸런서 (Layer 4)
- **GWLB (Gateway Load Balancer)**
 - 네트워크 보안 장비용 로드밸런서
 - ex. 방화벽
- **CLB (Classic Load Balancer)**
 - 과거에 사용

6.21 ELB (Elastic Load Balancer)

장점

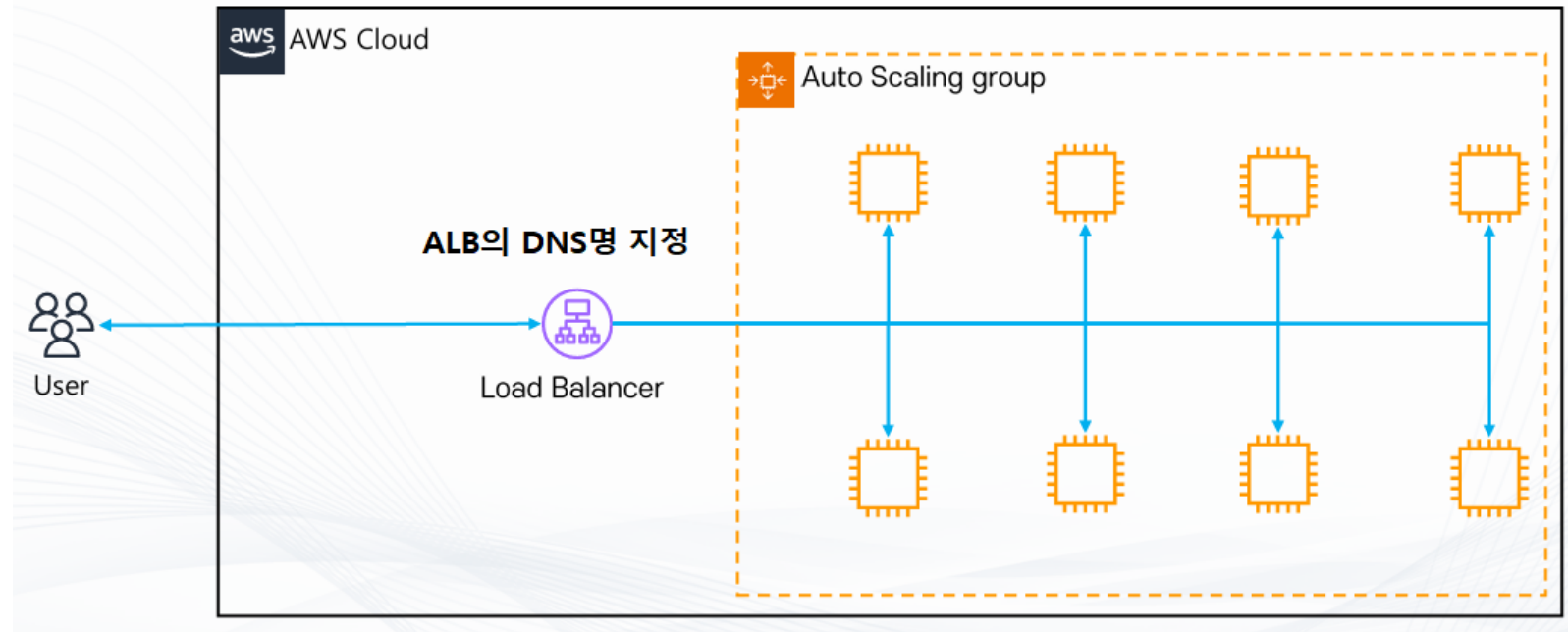
- 고가용성 (Multi AZ 지원)
- 자동 확장 (Auto Scaling 연동)
- 보안 (SSL 지원)
- 장애 대응 (Health Check)
- 트래픽 분산 (서버 부하 감소)

단점

- 추가 비용 (Load Balancer 비용)
- 설정 필요 (Target Group 등 구성 필요)
- 지연 가능성 (네트워크 hop 증가)

6.21 ELB (Elastic Load Balancer)

EC2 Auto Scaling과 연동



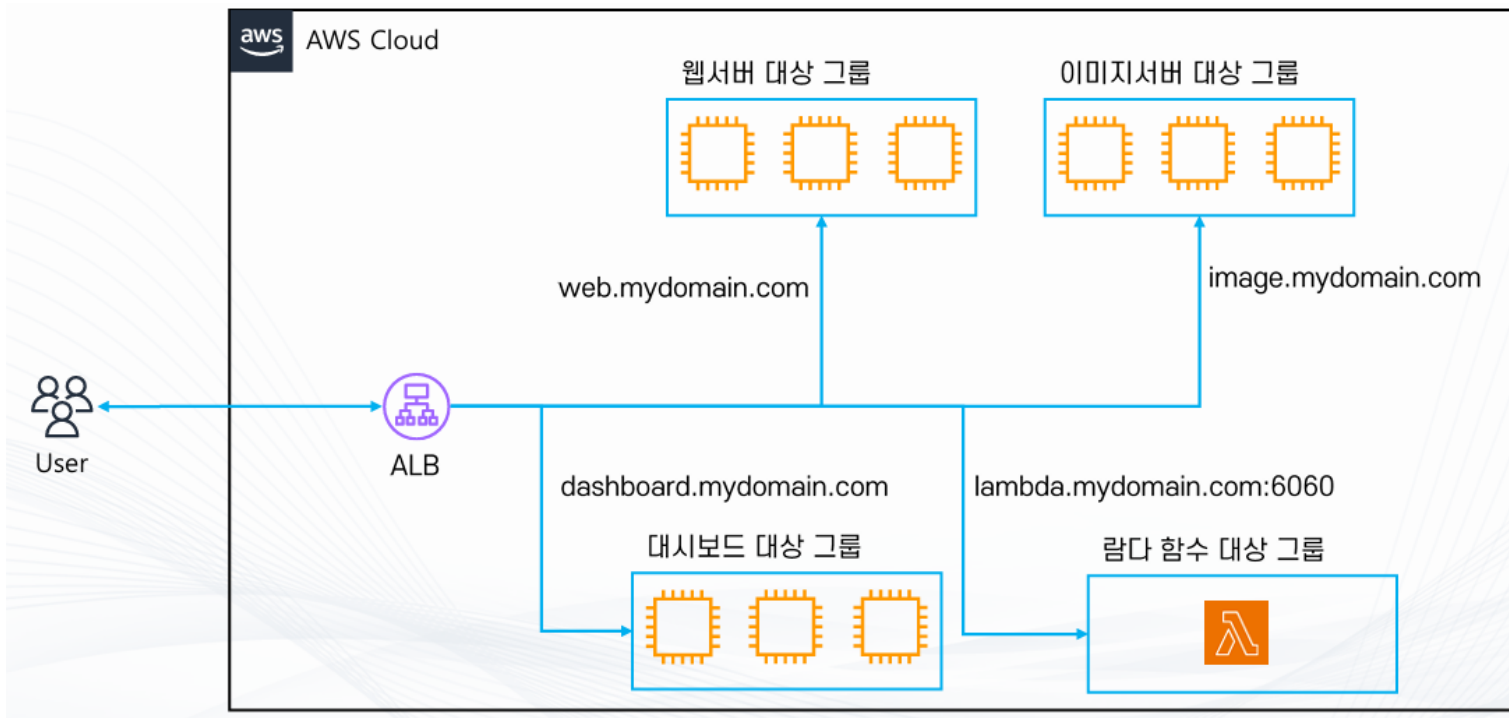
6.21 ELB (Elastic Load Balancer)

대상 그룹

- ELB가 라우팅 할 대상의 집합
- 구성
 - 대상 종류
 - Instance
 - IP
 - Lambda
 - ALB
 - 프로토콜
 - HTTP, HTTPS, gRPC, TCP 등
 - 기타 설정
 - 트래픽 분산 알고리즘, 고정 세션등

6.21 ELB (Elastic Load Balancer)

대상 그룹



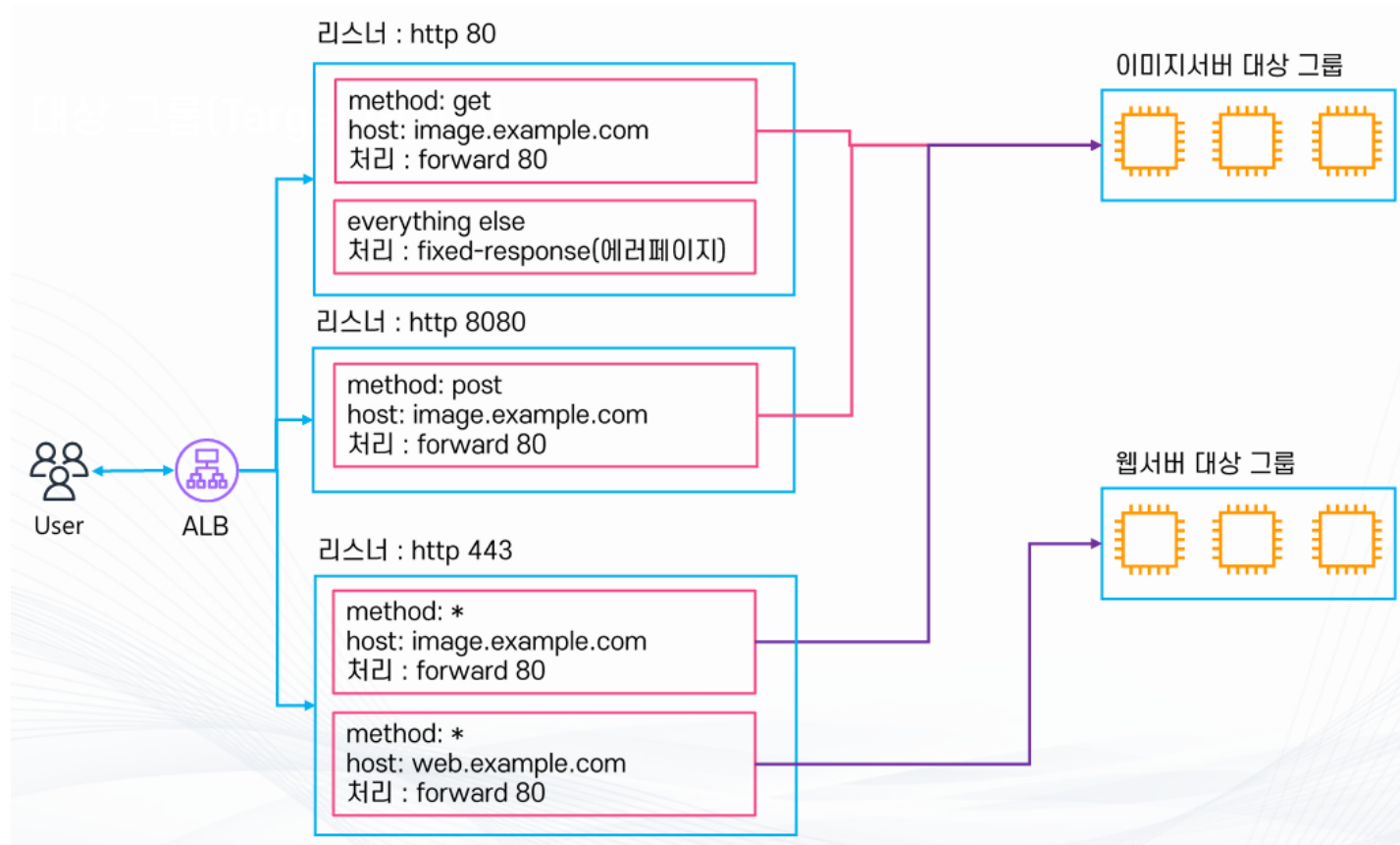
6.21 ELB (Elastic Load Balancer)

리스너

- ALB로 들어오는 요청을 처리하는 주체
 - 들어오는 트래픽의 프로토콜 + Port 단위
- 규칙(Rule)으로 ALB에서 어떤 요청을 받을지, 요청을 어디로 라우팅할지 결정
 - ex. http: 8080 포트로 트래픽을 받아서 A 대상그룹의 80 포트로 배분
- 규칙을 활용해서 다양한 조건에 따라 트래픽 배분 가능
 - ex. Header, QueryString, source IP, Method 등
- 들어온 트래픽 처리 방식
 - forward
 - redirect
 - fixed-response 등

6.21 ELB (Elastic Load Balancer)

리스너



6.22 실습

ALB + Autoscaling

- EC2 인스턴스 최소 2개 생성
 - 앞서 실습에서 사용했던 Auto Scaling Group 사용
- ELB 생성
 - EC2 > 로드 밸런서 > 로드밸런서 생성 클릭 > ALB 선택
 - 이름: demo-my-alb
 - 네트워크 매핑: 가용영역 및 서브넷 모두 선택
 - 보안 그룹: 새로운 보안그룹 생성
 - 보안그룹이름: demo-my-sg-alb
 - 인바운드 규칙: HTTP / 80, 0.0.0.0/0
 - 보안 그룹을 로드밸런서에 배정 (demo-my-sg-alb)
 - 대상 그룹 생성
 - 대상유형: 인스턴스
 - 대상그룹이름: demo-my-tg-alb
 - 대상 그룹 등록 및 생성
(모두 선택해서 포함시킴)
 - 대상 그룹 추가
 - 로드밸런서 생성

6.22 실습

로드 밸런서 (1/1) 새로운 기능

Elastic Load Balancing은 수신 트래픽의 변화에 따라 자동으로 로드 밸런서 용량을 확장합니다.

Q 로드 밸런서 필터링

이름	상태	유형	체계	IP 주소 유형
☒ demo-my-alb	☒ 활성	application	Internet-facing	IPv4

■ 웹 브라우저에서 요청

- 로드밸런서 > DNS 이름 이용
- 요청할 때마다 인스턴스 private IP가 바뀜

■ 임의로 하나의 인스턴스를 중지

- ALB가 Health Check하고 장애가 있는 인스턴스에는 더 이상 분배 안함
- ELB는 대상이 정상인지 비정상인지도 파악이 가능
- Auto Scaling에 의해서 자동으로 실행됨 (다시 시작했을 때 자동으로 대상그룹에 등록이 안됨)

■ 로드밸런서 삭제

■ Auto Scale Group 삭제

■ 대상 그룹 삭제

수고하셨습니다.